

Truth Tables and Applications of Logical Connectives

A 1.5-2 hour lecture on complete Boolean evaluation, truth-table construction, equivalence and Python verification.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Why Truth Tables?

A truth table systematically lists all possible truth values of component propositions. With n independent propositions there are 2^n rows.

Truth tables are useful for testing requirements, checking digital logic, verifying Boolean rules, and finding counterexamples to claimed equivalences.

2. Basic Truth Tables

NOT reverses truth. AND is true only for TT. OR is false only for FF. Implication is false only for TF. Biconditional is true when both sides have equal truth values.

Students should be able to construct each table without memorizing isolated answers by applying the definition.

3. Building Compound Truth Tables

For an expression such as $(p \vee q) \wedge \neg q$, create columns for p , q , $p \vee q$, $\neg q$, and the final conjunction. Evaluate from the innermost operation outward.

For three variables there are eight rows.

4. Tautology, Contradiction and Contingency

A tautology is true on every row; a contradiction is false on every row; a contingency has both true and false rows.

Examples: $p \vee \neg p$ is a tautology; $p \wedge \neg p$ is a contradiction.

5. Logical Equivalence

Two expressions are logically equivalent when they have identical truth values for every possible assignment. Truth tables provide a direct verification method.

A familiar example is $p \rightarrow q$ being equivalent to $\neg p \vee q$.

6. XOR

Exclusive OR means exactly one of p and q is true. It can be written $(p \vee q) \wedge \neg(p \wedge q)$, or represented by $p \text{ XOR } q$ in programming contexts.

7. Python Truth-Table Generator

Python can enumerate Boolean assignments and calculate each column. This makes truth tables practical for larger classroom examples.

8. CS Case Study — Access Control

Rule: $\text{access} = (\text{employee} \wedge \text{authenticated}) \vee \text{emergency_override}$.

Solution: enumerate the four possible values of `employee` and `authenticated` for each value of `override`. If `emergency_override=True`, `access` is always `True`; otherwise both `employee` and `authenticated` must be `True`.

9. CS Case Study — Fraud Detection

Rule: $\text{flag} = (\text{amount_high} \wedge \text{location_new}) \vee (\text{device_unknown} \wedge \text{velocity_high})$.

Solution: a transaction is flagged whenever either suspicious pair is simultaneously true. For $h=\text{True}, n=\text{False}, u=\text{True}, v=\text{True}$, $\text{flag}=\text{True}$ because the second pair is true.

10. Truth Table Exercise with Python

Use `itertools.product([False, True], repeat=2)` to generate four combinations. Compute $p \rightarrow q$ as $(\text{not } p) \text{ or } q$ and compare the result with a hand-built truth table.

Python Laboratory

Python Activity 1

```
from itertools import product

print("p q | p->q")
for p, q in product([False, True], repeat=2):
    implication = (not p) or q
    print(p, q, implication)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
# Check a proposed equivalence: p -> q == not p or q
for p, q in product([False, True], repeat=2):
    left = (not p) or q
```

```
right = (not p) or q
assert left == right
print("Equivalent for all cases")
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Construct the truth table for $p \wedge q$.
2. Construct the truth table for $p \vee q$.
3. Construct the truth table for $p \rightarrow q$.
4. Construct the truth table for $p \leftrightarrow q$.
5. Construct the truth table for $(p \vee q) \wedge \neg q$.
6. Determine whether $p \vee \neg p$ is a tautology.
7. Determine whether $p \wedge \neg p$ is a contradiction.
8. Determine whether $p \rightarrow q$ and $\neg p \vee q$ are equivalent.
9. Construct a truth table for a three-variable CS access rule.
10. Use Python to generate all eight rows for p , q and r .
11. Write a Boolean expression for a rule that accepts exactly one of two authentication methods.
12. Explain why testing only a few truth assignments is not sufficient to prove equivalence.

End-of-Lecture Student Practice — No Solutions

1. Prove or disprove by truth table that $p \rightarrow q$ is equivalent to $\neg p \vee q$.
2. Construct a truth table for a three-variable access rule of your choice.
3. Use Python to verify the result.

Quick Recap

Truth tables give complete evaluation for all Boolean assignments.